Post-Quantum Cryptography for IoT and Lightweight Environments: Challenges, Algorithms, and the Road Ahead

Shayan Raza, Safiyan Aman, Muhammad Asad Siddiqui, Muhammad Hamza Abbas
Department of Computer Engineering
Habib University
Karachi, Pakistan

***Abstract*—With the rapid growth of IoT devices, a silent security crisis is in the offing that cannot be easily rectified once it sets in. Modern cryptographic protocols to protect connected systems, including RSA, ECDH, ECC, are based on mathematical problems that can be solved in a sufficiently large quantum computer in a time of order of magnitude. This threat is two times hard in the case of constrained IoT hardware, with its tight RAM budgets, limited processing power, and strict energy envelopes: such devices must eventually migrate to quantum-resistant implementations, yet many of such implementations are heavy enough to be infeasible on microcontrollers. To get a clear idea of the actual state of affairs, we surveyed recent work on post-quantum cryptography (PQC) in resource-constrained environments, including the four algorithms recently finalized by NIST in 2024, namely ML-KEM (CRYSTALS-Kyber), ML-DSA (CRYSTALS-Dilithium), FALCON, and SLH-DSA (SPHINCS+). We put the same practical questions to each of them: how much RAM does it need, how much flash, and what is its cost in energy? We also discuss ASCON, which was standardized separately by NIST as its response to lightweight symmetric cryptography, and discuss how it fits in with PQC to cover the symmetric layer. The survey has been based on empirical benchmarks of ARM Cortex-M microcontroller studies and Raspberry Pi platform tests, in the key sizes, signature sizes, memory footprints and the implementation headaches side-channel exposure, weak entropy sources, protocol overhead — that are actually encountered when you actually attempt to deploy these algorithms rather than just benchmark them in isolation. Where the numbers are good, we say so; where the numbers fall short of certain hardware levels, we say so, and we discuss what schemes of hybrid schemes, KEMTLS, and hardware acceleration can do with it that are realistically achievable.**

***Index Terms*—Post-Quantum Cryptography, Internet of Things, Lightweight Cryptography, ASCON, CRYSTALS-Kyber, ML-KEM, CRYSTALS-Dilithium, ML-DSA, FALCON, SPHINCS+, SLH-DSA, NIST Standardization, Constrained Devices, Side-Channel Attacks, KEMTLS**

I. Introduction

The world of embedded systems is more than most of the individuals in that world are ready to handle. In 1994, Shor demonstrated that a quantum computer large enough could factor integer numbers and compute discrete logarithms in the same time as any known algorithm, i.e. in a time that is polynomially-bounded [1]. Over an extended period of time this has felt like something that is a long way away; to create a quantum machine that is powerful enough to break 2048-bit RSA, appeared to be a problem that researchers would not be able to solve until several decades had passed. That is a window that has been closing. Even the threat estimates by NIST now indicate that RSA will become insecure by 2030, and that hardware advances at IBM, Google, and others continue to push the goalposts [3].

Where this gets particularly uncomfortable is in the IoT space. By 2022, connected IoT devices worldwide had crossed 14 billion and the number keeps growing [9]. Buried in that count are industrial sensors, pacemakers, smart meters, vehicle control units, RFID tags — devices that were never designed to receive cryptographic firmware upgrades and that routinely stay in service for ten to fifteen years. A sensor node shipped today with ECDH key exchange baked in could still be running that exact code in 2035. That is the crux of the problem: the cryptographic choices we make for constrained IoT hardware today are commitments that outlast the threat landscape we can currently see.

Making things worse is the "store now, decrypt later" (SNDL) threat. An adversary does not need a quantum computer right now to begin attacking today's communications — they just need storage. Intercepted ciphertext recorded today can sit in an archive until a capable quantum machine becomes available to break it. With long-lived sensitive information, such as medical records, grid control telemetry, classified government traffic, it is not a hypothetical risk [9]. The practical implication here is that migration to post-quantum cryptography should occur prior to the arrival of quantum hardware and not afterward.

In recognition of this, NIST ran a multi-year public standardization competition which concluded its initial phase in August 2024 and published three finalized post-quantum standards: FIPS 203 (ML-KEM, based on CRYSTALS-Kyber) to encapsulate keys, FIPS 204 (ML-DSA, based on CRYSTALS-Dilithium) to sign and verify digital signatures, and FIPS 205 (SLH-DSA, based on SPHINCS+) to sign and verify hash-based signatures [5]–[7]. Another fourth algorithm called FALCON is yet to be published as FIPS 206. In March 2025, NIST additionally selected HQC as a code-based KEM alternative following a separate fourth-round evaluation [4]. On the symmetric side, NIST concluded its separate lightweight cryptography competition in February 2023 by selecting the ASCON family, subsequently formalizing it as NIST SP 800-

232 in August 2025 [8]. Together these two tracks represent the most comprehensive government-led effort to quantum-proof both the asymmetric and symmetric layers of communication for constrained devices.

This paper provides a structured survey of how those algorithms perform on IoT-class hardware, what implementation obstacles arise in practice, and how ASCON fits alongside PQC to form a complete quantum-resistant security stack. The rest of the paper is organized in such a way that Section 3 discusses related work found in literature, followed by a discussion in Section 4 and the paper is concluded in Section 5.

II. Background

A. The Quantum Threat to Classical Cryptography

Today's public-key infrastructure is built on two hard mathematical problems: integer factorization (RSA) and the discrete logarithm problem (ECC, ECDH, DSA). Shor's algorithm solves both in $O((\log N)^3)$ gate operations on a quantum computer, which would completely break these schemes at any practically used key size [1]. Symmetric primitives like AES face a weaker threat: Grover's algorithm provides a quadratic speedup for exhaustive search, effectively halving the security level so that AES-128 provides roughly 64 bits of post-quantum security. Doubling key lengths — using AES-256 rather than AES-128 — is sufficient to recover the full security margin. For public-key systems, no analogous key-size fix exists; the underlying mathematical problems simply collapse under Shor, and entirely new algorithmic foundations are required [2].

B. Resource Constraints in IoT Devices

IoT devices span an enormous capability range, but the lower tiers — which represent the bulk of deployed units — are severely constrained. ARM Cortex-M4 class processors (e.g., STM32F407) operate at up to 168 MHz with 192 KB of SRAM and 1 MB of flash. The Cortex-M0 tier, common in sensor nodes and RFID readers, may provide only 8–32 KB of RAM. Many of these devices operate from coin cells or energy harvesting, so cryptographic operations that are merely slow on a server can become energy-prohibitive on embedded hardware [11]. Three resource dimensions dominate feasibility analysis:

- **RAM**: determines whether key material and intermediate computation buffers fit without running out of dynamic memory.
- **ROM/Flash**: determines whether algorithm code and lookup tables can be stored alongside the application.
- **Energy**: determines whether a cryptographic operation is affordable within the device's power budget, particularly for battery-constrained nodes.

Public-key and signature sizes also matter for bandwidth-constrained protocols such as LoRaWAN and Zigbee, where individual packets may be limited to a few hundred bytes.

C. PQC Algorithm Families

NIST's selected algorithms draw from two primary mathematical families. The lattice-based schemes — ML-KEM, ML-DSA, and FALCON — rely on the hardness of the Module Learning With Errors (MLWE) problem and the NTRU problem respectively, both of which are believed to resist quantum attacks. The hash-based scheme, SLH-DSA (SPHINCS+), relies solely on the collision resistance and preimage resistance of the underlying hash function, giving it the most conservative and least assumption-dependent security argument in the portfolio [2]. HQC, the fourth-round addition, relies on the hardness of decoding random quasi-cyclic codes [4].

III. Related Work / Literature Review

A. Lattice-Based PQC on Constrained IoT Hardware

1) ML-KEM (CRYSTALS-Kyber): CRYSTALS-Kyber, standardized as ML-KEM under FIPS 203, is a key encapsulation mechanism built on the Module Learning With Errors (MLWE) problem. Public keys sit between roughly 800 bytes at the lowest security level (ML-KEM-512) and 1,568 bytes at the highest (ML-KEM-1024), with encapsulated ciphertexts of similar sizes [5]. By post-quantum standards these are genuinely compact numbers, which is a large part of why ML-KEM has attracted more serious IoT deployment interest than any other algorithm in the NIST portfolio.

Lopez *et al.* put this to the test by running Kyber, BIKE, and HQC on Raspberry Pi hardware using the Open Quantum Safe `liboqs` library alongside mbedTLS [10]. Their findings are one of the most practically applicable empirical data sets that can be applied to real IoT settings. Kyber emerged a winner on both latency and memory in all the operations tested. BIKE and HQC — the code-based candidates — were both evidently perceptively heavier on both counts, to reinforce the pragmatic case of Kyber on anything bandwidth-constrained or compute-limited.

On the microcontroller side, the pqm4 project offers the most systematically detailed view of what such algorithms cost on real embedded hardware. Kannwischer *et al.* estimate the stack requirement of ML-KEM-512 at approximately 16 KB key generation and encapsulation at the 128-bit security level [12] — tight but workable on a mid-range Cortex-M4, unless the application is already heavily consuming its RAM. One of the key reasons why Kyber can achieve this level of efficiency is the Number Theoretic Transform (NTT), which manages the multiplication of polynomials in a much more efficient manner as compared to a simple schoolbook method. An implementation that takes advantage of the DSP multiply-accumulate instructions and Montgomery reduction cut cycles makes the code count significantly lower than that of generic C code [13].

Kyber has undergone realistic protocol stress-tests as well. Tschofenig and Kampanakis experimented with PQC-extended mbedTLS handshakes on IoT edge devices and found Kyber workable but not free, the latency and energy overhead

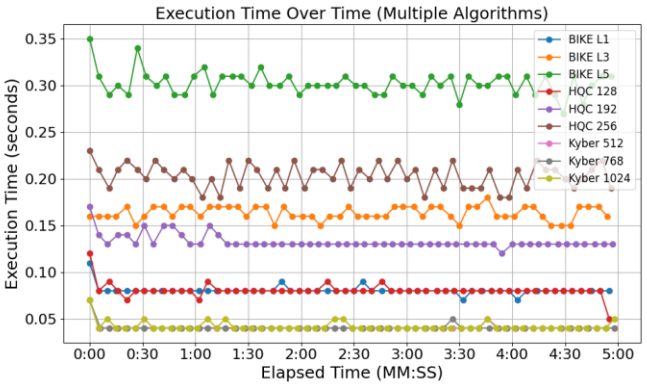

Fig. 1. Execution time over time for BIKE, HQC, and Kyber.

TABLE I
EXECUTION TIME (IN SECONDS) FOR POST-QUANTUM CRYPTOGRAPHY ALGORITHMS.

PQC Algorithm	Execution Time
BIKE L1	0.081 ± 0.005
BIKE L3	0.164 ± 0.007
BIKE L5	0.302 ± 0.013
HQC 128	0.081 ± 0.007
HQC 192	0.133 ± 0.008
HQC 256	0.203 ± 0.013
Kyber 512	0.041 ± 0.004
Kyber 768	0.041 ± 0.004
Kyber 1024	0.042 ± 0.005

TABLE II
POWER CONSUMPTION (WATTS) OF PQC ALGORITHMS.

PQC Algorithm	Low		High	
	Client	Server	Client	Server
BIKE L1	3.2	3.5	3.5	4.7
BIKE L3	3.2	3.5	3.5	4.8
BIKE L5	3.2	3.5	3.5	5.2
HQC 128	3.2	3.4	3.6	4.0
HQC 192	3.2	3.5	3.6	4.1
HQC 256	3.2	3.5	3.7	4.7
Kyber 512	3.2	3.4	3.5	3.9
Kyber 768	3.2	3.5	3.5	3.9
Kyber 1024	3.2	3.5	3.5	3.9

Fig. 2. Mean execution time (Table I) and power consumption (Table II) across security levels.

Fig. 3. Performance evaluation of PQC KEMs on resource-constrained IoT hardware. Kyber variants consistently achieve the lowest execution times (Kyber 512 at 0.041s vs BIKE L5 at 0.302s) with comparable power draw across all algorithms. Reproduced from [10].

compared to ECDH was mainly caused by the larger key material being crossed the wire, rather than by the computation itself [11]. On the industry side, Google shipped hybrid post-quantum key exchange (X25519 paired with ML-KEM) as the default in Chrome 124 in April 2024 [22] — a significant indication that PQC is entering production, even though embedded IoT stacks lag far behind the uptake of browsing.

2) ML-DSA (CRYSTALS-Dilithium): The workhorse of the NIST portfolio in the post-quantum digital signatures is ML-DSA, based on CRYSTALS-Dilithium (which in turn is based on the NIST-standardized FIPS 204). It has the same MLWE mathematical base as ML-KEM, except that the design priority is no longer compactness, but implementation robustness [6]. That trade-off appears in the numbers: signatures are run that are between about 2,420 bytes at ML-DSA-44 and 4,627 bytes at ML-DSA-87, and public keys are between about 1,312 bytes and 2,592 bytes.

Those sizes matter in the case of the IoT where even a small increase in the size of a packet is translated into real energy consumption. According to pqm4 benchmarking, the signing path alone of Dilithium will consume approximately 40–60 KB of stack on a 128-bit security level Cortex-M4 [12]. This is more than the total amount of available RAM on the lowest cost category of Cortex-M0 hardware alone, effectively eliminating ML-DSA as a viable solution to the least expensive tier of devices not having hardware acceleration. On mid-range Cortex-M4 hardware it is viable, but only for operations that happen infrequently — authenticating a firmware image or issuing a certificate, not signing individual packets.

One practical advantage of ML-DSA for embedded deployments is its support for deterministic signing. Many IoT microcontrollers lack a high-quality hardware random number generator, which creates problems for signing schemes that require fresh randomness per signature. Dilithium's deterministic mode eliminates that dependency during signing, relying instead on a pseudorandom function of the private key and message, substantially simplifying its deployment on entropy-poor hardware.

To make the ML-DSA versus FALCON trade-off concrete, consider an over-the-air (OTA) firmware update scenario — one of the most common asymmetric signing use cases in IoT. A build server signs a 256 KB firmware binary once; thousands of field devices verify the signature before applying the update. In this workflow, signing cost is irrelevant (it happens once, offline, on powerful hardware), but signature size directly affects the OTA payload and therefore transmission energy. FALCON-512's ≈666 byte signature is 3.6× smaller than ML-DSA-44's 2,420 bytes, translating to roughly 1.8 KB saved per update over LoRaWAN — a meaningful saving when link budget is scarce and devices are battery-powered. Both algorithms can be run quickly on a Cortex-M4 to the extent that the verification time difference between the two algorithms is insignificant compared to the cost of transmitting the firmware over the air. In other words: when it comes to OTA updates, one would pick FALCON to save on the connection, and when it comes to frequent on-device authentication, where the absence of the M4 of a double-precision FPU is the bottleneck, one would pick ML-DSA to be the more deployable option.

3) FALCON: FALCON uses an alternate lattice scheme, based on the NTRU problem, with the FN-DSA construction and with a standardization effort underway as FIPS 206. The difference is signature size: FALCON-512 generates signatures of approximately 666 bytes, compared to 2,420 bytes — about 3.6× smaller — generated by ML-DSA-44. That is not an academic gap, though, over LoRa or narrowband cellular connections where each bit costs battery life.

The catch is in how signatures are generated. FALCON's signing algorithm requires Gaussian sampling over polynomial lattices using double-precision (53-bit mantissa) floating-point arithmetic. Most IoT microcontrollers either have no FPU at all or only a single-precision one, so double-precision has to be emulated entirely in software — and that emulation is expensive in both clock cycles and code size.

Benchmark results from Hamouda *et al.* [14] make this contrast concrete, as also shown in Fig. 4. On the Cortex-M4, FALCON signing is substantially slower than Dilithium's because every double-precision operation goes through the software emulation path. Switch to the Cortex-M7 — the only Cortex-M processor with a native 64-bit FPU — and FALCON signing speeds up by 6.2–8.3× in clock cycles and 6.2–11.8× in wall time. Dilithium, which needs no floating-point at all, barely improves from M4 to M7 beyond the raw clock speed difference.

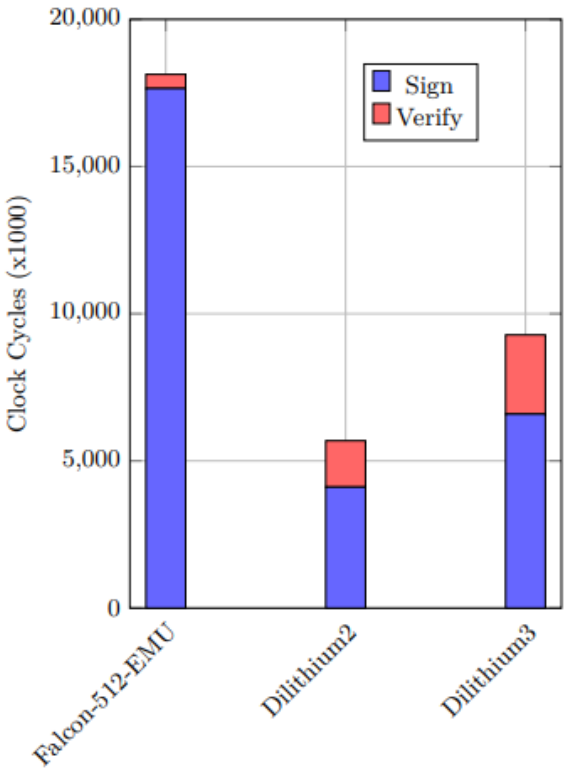

Fig. 4. Signing and verification benchmark comparison between ML-DSA (Dilithium) and FALCON on the ARM Cortex-M4 and Cortex-M7. FALCON gains 6.2–8.3× in clock cycles on the M7 due to its native double-precision FPU, while Dilithium's improvement is modest. This illustrates why FALCON is poorly suited for M4-class IoT hardware without a hardware FPU. Reproduced from [14].

Separately, Pornin demonstrated that careful memory layout optimization on the Cortex-M4 can reduce FALCON's dynamic memory consumption by 43% compared to its reference implementation, bringing the key generation time down to approximately 682 ms and signing to 479 ms at n=512 [15]. Verification is considerably faster: Choi *et al.* achieved over 68 verifications per second for FALCON-512 on a Cortex-M4 using handcrafted assembly, indicating that verification at least is quite practical even on embedded hardware [16]. The implication for system designers is clear: FALCON should be considered for IoT hardware that either provides a Cortex-M7 or better for signing, or where signing happens off-device (e.g., signing firmware images on a server) and only verification runs on the embedded platform.

B. Hash-Based Signatures: SLH-DSA (SPHINCS+)

SLH-DSA, standardized under FIPS 205, takes a qualitatively different approach from the lattice-based schemes. Rather than resting on a structured algebraic problem, its security depends entirely on the preimage resistance and collision resistance of an underlying hash function — SHA-256, SHA-512, or SHAKE depending on the parameter set [7]. That is the most conservative security argument in the NIST PQC portfolio: a future breakthrough against lattice mathematics would leave SLH-DSA completely untouched, as long as the hash function itself holds.

The cost of this conservatism is severe for IoT deployments. SLH-DSA signatures range from approximately 7,856 bytes in the fast-parameter variant (SLH-DSA-SHAKE-128f) to 49,856 bytes in the small variant (SLH-DSA-SHA2-256s). Signing involves evaluating a large hypertree of Winternitz OTS and FORS tree operations, which on desktop hardware already takes hundreds of milliseconds and would be impractical on constrained microcontrollers for any application requiring frequent signatures [9].

Where SLH-DSA does remain relevant for IoT is in asymmetric deployment: signing performed offline on a workstation or server, and only verification executed on the embedded device. Verification requires far fewer hash evaluations than signing, and the code footprint for verification-only deployment is small enough to fit on modest platforms. This makes SLH-DSA a realistic option for firmware update authentication — where a build server signs the firmware image once, and thousands of devices each verify it independently — particularly in applications where the conservative, hash-only security argument is required by policy or regulation [9], [17].

C. ASCON: Lightweight Authenticated Encryption

1) Design and Standardization: ASCON was developed in 2014 by Dobraunig, Eichlseder, Mendel, and Schläffer at Graz University of Technology and Infineon Technologies [18]. After winning the CAESAR competition for lightweight authenticated encryption in 2019, it went on to win NIST's lightweight cryptography standardization competition in February 2023, and was subsequently formalized as NIST SP 800-232 in August 2025 [8]. The standard defines four main primitives: ASCON-AEAD128 for authenticated encryption with associated data (AEAD), ASCON-Hash256 as a lightweight alternative to SHA-3, ASCON-XOF128 and ASCON-CXOF128 as extendable output functions.

ASCON's architecture is based on a duplex sponge construction operating over a 320-bit internal state organized as five 64-bit words, illustrated in Fig. 5. The authenticated

encryption process passes through four sequential stages: an initialization phase that mixes the key and nonce into the state, an associated data absorption phase, a plaintext encryption phase where ciphertext blocks are extracted, and a finalization phase that produces the authentication tag [17]. Between each block, the internal state is updated by the ASCON permutation, an iterated substitution-permutation network (SPN) operating over the 5-word state with a 5-bit S-box applied 64 times in parallel and a linear diffusion layer. The number of permutation rounds differs between the initialization and finalization phases ($p^a = 12$ rounds) and the intermediate processing phases ($p^b = 6$ or 8 rounds depending on variant).

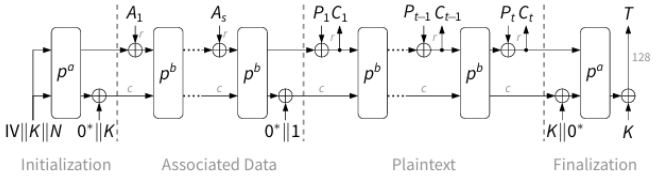

Fig. 5. ASCON authenticated encryption process with associated data, illustrating the four-stage duplex sponge construction: initialization, associated data absorption, plaintext encryption, and tag finalization. The permutations p^a and p^b operate on the 320-bit internal state. Reproduced from [17].

2) Performance on Constrained Hardware: ASCON's hardware numbers are genuinely remarkable. The ASCON-128 AEAD variant fits in as few as 2,000 gate equivalents on an ASIC [19] — a figure that puts it within reach of RFID tags and implantable sensors that would choke on AES-GCM. On software platforms the 320-bit state sits comfortably within the general-purpose register file of any 32-bit or 64-bit processor, so bitsliced implementations run without needing any special instructions.

Svensén *et al.* ran ASCON head-to-head against eSTREAM finalists and several modern stream ciphers on the Nordic Thingy:53, a Bluetooth mesh development board, across both low-data-rate and high-throughput test scenarios [20]. ASCON-32 — the 32-bit software variant — beat the field on bulk throughput and remained competitive in the Bluetooth mesh workload. One design aspect that rarely gets enough attention is ASCON's built-in friendliness to side-channel countermeasures. In the SP 800-232 documentation, NIST specifically called out that ASCON's permutation structure is easier to mask than AES, which requires significantly more complex and silicon-hungry protection schemes [8]. For IoT devices that may be physically accessible to attackers, this is a meaningful advantage.

The first complete survey of ASCON following the 2023 standardization, including both FPGA and ASIC implementations as well as software profiles on microcontrollers [17]. They found that although ASCON does not optimize every single benchmark, the permutation-based sponge architecture lets designers tune the throughput-area-power trade-off with a flexibility currently simply not afforded by block-cipher designs such as AES-GCM.

3) ASCON Hash Functions for IoT Security Primitives: In addition to authenticated encryption, the ASCON hash variants have been the focus of an increasing research interest to expand the use of the hash variants in broader IoT security applications. A benchmarking study of ASCON-Hash256, ASCON-XOF128 and ASCON-CXOF128 compared their suitability in Bloom filter-based replay-attack detection, Merkle tree aggregation to support data integrity, lightweight device fingerprinting and tamper-evident logging — all the components often needed in the IoT security architecture [21]. The sponge structure enables streaming operation that maps naturally onto constrained hardware with limited RAM, since the entire message does not need to be buffered before processing begins.

D. Hybrid PQC and Protocol Integration

1) Hybrid Classical + Post-Quantum Key Exchange: A widely adopted transitional strategy is the hybrid approach: combining a classical key exchange (e.g., X25519) with a post-quantum KEM (typically ML-KEM) in a single handshake such that the session is secure if either component holds. This provides protection against SNDL attacks through the PQC component while retaining classical security in the event that PQC assumptions are later found to be weaker than believed.

Gómez-Cambronero *et al.* studied TLS 1.3 handshakes under three configurations — classical X25519 only, hybrid X25519+ML-KEM, and pure ML-KEM — in a controlled load-test environment of up to 100 transactions per second [22]. Their layered analysis, reproduced in Fig. 6, breaks the total transaction latency into five phases: TCP handshake, TCP-to-TLS transition, TLS handshake, TLS-to-application transition, and HTTP response. What the results show is that PQC overhead lands almost entirely in the TLS handshake layer, and the culprit is data volume — larger key material crossing the wire — rather than computation time on either end. Pure ML-KEM tracked the hybrid configuration closely in overall latency, and both sat only modestly above classical X25519 in a wired, low-latency test environment.

For IoT links this changes considerably. LoRa, NB-IoT, and Zigbee are not wired Ethernet — and even in that wired test the dominant cost was data size. Over a LoRa link with a 242-byte maximum payload, ML-KEM-512's 768-byte ciphertext alone has to be split across at least four packets, with each fragmentation boundary adding a round-trip and the energy cost of additional transmissions. Hybrid PQC slots in reasonably cleanly for gateway-to-cloud TLS, but hits a real wall at the innermost wireless hop, where compact protocol designs or KEMTLS adaptations are needed before PQC integration is truly seamless.

2) KEMTLS: Eliminating Server Signatures from Handshakes: An important protocol-level optimization is KEMTLS, a variant of TLS 1.3 that replaces the server's digital signature during authentication with a KEM-based implicit authentication step. By using ML-KEM for both key establishment and server authentication, KEMTLS avoids generating a PQC signature during the handshake entirely, which is the most expensive PQC operation in conventional TLS. Liu *et al.*

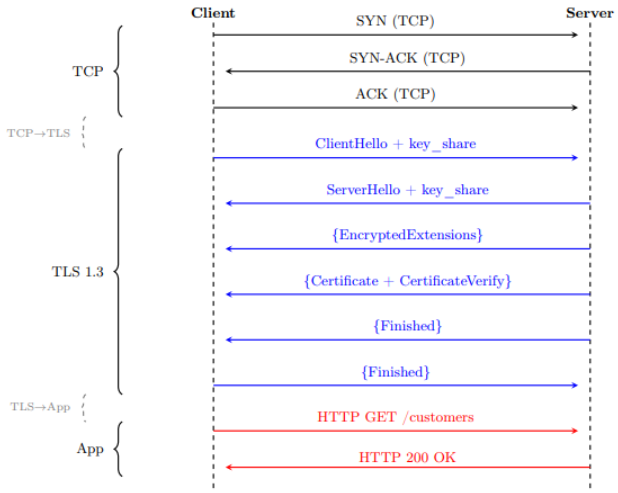

Fig. 6. Five-layer latency decomposition of TLS 1.3 transactions under classical (X25519), hybrid (X25519+ML-KEM), and pure post-quantum (ML-KEM) key exchange, at up to 100 transactions per second. PQC-induced overhead is concentrated in the TLS handshake layer and is primarily caused by larger key material, not computation. Reproduced from [22].

found that compared to post-quantum TLS with conventional signatures, KEMTLS reduces handshake latency by up to 38% and also achieves lower peak memory usage, since KEM operations require less stack than signature generation [9]. For IoT TLS stacks where memory is the primary constraint, this makes KEMTLS a compelling design target.

3) Hardware Acceleration: Software-only PQC on microcontrollers will always face limits. A growing body of work therefore explores hardware accelerators to make PQC viable even on more constrained devices. The PQC-HA framework from TU Darmstadt provides a structured approach to synthesizing hardware accelerator cores for ML-KEM and ML-DSA from C reference implementations using High-Level Synthesis, targeting FPGAs via the TaPaSCo infrastructure [23]. The framework demonstrated that HLS-generated hardware substantially outperforms software-only implementations across all security levels, while remaining portable across FPGA families.

For FALCON specifically, hardware acceleration of the FFT/IFFT core — which dominates FALCON's arithmetic — resolves the floating-point problem entirely, since dedicated digital signal processing logic in hardware can perform the required operations without relying on a software FPU [24]. This confirms that FALCON's embedded deployment problem is not inherent to the algorithm; it is a hardware availability problem that dedicated silicon can solve.

IV. Discussion

A. Mapping Algorithms to IoT Device Tiers

IoT hardware is not a single category. Table I — presented at the end of this section — maps each algorithm against three representative hardware tiers and provides quantitative parameters drawn from the literature. The values for key and signature sizes are taken from the NIST FIPS documents [5]–[7], and the stack estimates are derived from pqm4 benchmarking data [12], [14]. Note that Table I is distinct from the performance tables embedded within Fig. 3; the former covers all algorithms across resource dimensions, while the latter reports empirical timing and power measurements for the three KEMs evaluated by Lopez *et al.*

Reading across the table, a few patterns are clear. ML-KEM is the most broadly deployable PQC primitive: its stack is within reach of mid-range Cortex-M4 platforms, and its key and ciphertext sizes, while larger than ECDH, are manageable for most network protocols. ML-DSA is viable on M4-class hardware for low-frequency signing tasks — firmware authentication is the ideal use case — but its stack demand rules out lower-end hardware without dedicated support. FALCON offers the most compact signatures in the portfolio and is therefore appealing for bandwidth-limited links, but signing on M4-class hardware is slow enough to be impractical for any time-sensitive application; it genuinely requires the M7 or a hardware accelerator. SLH-DSA's enormous signatures and slow signing effectively restrict it to offline signing workflows where a server generates the signature and devices only verify.

ASCON sits in a qualitatively different position from all the PQC algorithms. As a symmetric primitive it does not participate in key establishment, but it provides the authenticated encryption and integrity guarantees that account for the bulk of per-packet cryptographic overhead in IoT communications. Its sub-kilobyte RAM requirement and 2,000 gate-equivalent hardware footprint make it accessible to every IoT hardware tier. Importantly, it does not compete but rather complements the PQC algorithms: ML-KEM is used to establish a common secret, and then that secret can be used to protect all the following data traffic by ASCON-AEAD128.

B. Implementation Challenges

Several practical challenges complicate PQC deployment beyond the raw performance figures.

Side-channel vulnerability. Without careful design of implementation, lattice-based schemes can leak private key material either through power consumption or timing channels. The sampling of ML-DSA (rejection sampling) and FALCON (Gaussian sampling) are particularly sensitive. Any device that can be physically vulnerable to an opponent, but that imposes code complexity and in some cases even extra cycle overhead is known as constant-time implementations. The design of ASCON clearly supports masking countermeasures far easier than AES, making it a better option on the symmetric layer that devices in physically hostile environments are deployed in [17], [19].

Entropy scarcity. Key generation for all PQC algorithms requires high-quality random numbers. Many IoT microcontrollers have weak or absent hardware random number generators. This is a well-known embedded systems problem but PQC makes it more acute: lattice schemes that sample error polynomials from Gaussian distributions require large quantities of random bits per operation. Deterministic key

TABLE I

COMPARATIVE SUMMARY OF NIST PQC ALGORITHMS FOR CONSTRAINED IOT ENVIRONMENTS (SEE ALSO FIGS. 3 AND 4 FOR EMPIRICAL BENCHMARKS SUPPORTING THESE ESTIMATES). STACK FIGURES ARE FOR THE MOST DEMANDING OPERATION AT THE LOWEST NIST SECURITY LEVEL; SUITABILITY RATINGS REFLECT SOFTWARE-ONLY DEPLOYMENT WITHOUT HARDWARE ACCELERATION.

Algorithm	Type	Pub. Key	Sig./CT Size	Est. Stack	Cortex-M0	Cortex-M4/M7
ML-KEM-512	KEM	800 B	768 B (CT)	≈16 KB	Marginal	Yes
ML-KEM-768	KEM	1,184 B	1,088 B (CT)	≈24 KB	No	Yes
ML-DSA-44	Sign	1,312 B	2,420 B	≈48 KB	No	Yes
FALCON-512	Sign	897 B	≈666 B	≈30 KB	No	M7 preferred†
SLH-DSA-128f	Sign	32 B	17,088 B	<5 KB (verify)	Verify only	Verify only
ASCON-AEAD128	AEAD	128-bit key	16 B tag	<1 KB	Yes	Yes

† FALCON signing requires 53-bit double-precision arithmetic; M4 requires costly software emulation. M7's native FPU makes signing feasible.

derivation from a well-seeded hardware entropy source is the practical solution, and ML-DSA's deterministic signing mode specifically removes the randomness requirement from the signing path [6].

Protocol overhead. The impact of PQC does not stop at the cryptographic operation itself. Larger key material and signatures propagate into certificate sizes, TLS record sizes, and OTA update payloads. In LoRaWAN, where the maximum application payload per packet can be as low as 51 bytes depending on the spreading factor, even ML-KEM-512's 768-byte ciphertext would require fragmentation across more than fifteen packets. Protocol adaptations — compressed certificate formats, session resumption, KEMTLS — are therefore essential engineering work, not optional optimizations [9], [22].

Long device lifetimes and update constraints. Many IoT devices cannot be easily firmware-updated after deployment. Industrial sensors embedded in concrete, medical implants, and infrastructure meters may go years without a software update. This means that PQC support must be built into new devices at design time rather than added later. The hybrid approach provides a transitional bridge but does not eliminate the need for forward planning in hardware and software architecture.

C. ASCON and PQC as a Unified Security Stack

A practical point that the literature sometimes underemphasizes is that ASCON and the NIST PQC algorithms address different layers of the security stack and are best understood as complementary. ASCON handles authenticated symmetric encryption — fast, tiny, and quantum-resistant by virtue of its 128-bit key size being robust against Grover's quadratic speedup. The PQC algorithms handle asymmetric operations: key establishment (ML-KEM), digital authentication of code or certificates (ML-DSA, FALCON), and conservative long-lived signing (SLH-DSA).

A complete quantum-resistant IoT security architecture therefore looks like this: ML-KEM performs the key exchange, delivering a shared secret; ASCON-AEAD128 consumes that secret to protect all subsequent session traffic. For code signing, ML-DSA or FALCON signs firmware images on a build server, and the embedded device runs verification only. ASCON-Hash256 can replace SHA-256 for any device-side hashing needs. This layered design is achievable today on mid-range Cortex-M4 hardware using open-source libraries such as `liboqs` and the ASCON reference implementation.

D. Open Research Directions

Several threads in the literature remain insufficiently addressed. Cortex-M0 PQC deployment has received almost no attention: the constraints of 32 KB RAM devices likely require hardware/software co-design rather than software-only optimization. KEMTLS for embedded TLS stacks has been analytically validated but comprehensive implementation benchmarks on mbedTLS and wolfSSL are still sparse. Full end-to-end benchmarks combining a PQC KEM, ASCON at the symmetric layer, and a realistic IoT protocol stack (DTLS, CoAP, MQTT-SN) do not yet exist in the open literature and would substantially advance the field. Finally, post-quantum certificate chain compression — reducing the bandwidth overhead of transmitting PQC-signed X.509 certificates — remains an active problem with no settled solution.

V. CONCLUSION

Post-quantum cryptography has moved from a theoretical imperative to an engineering reality. Between the NIST PQC standards finalized in 2024 and the ASCON lightweight cryptography standard published in 2025, the building blocks now exist for a complete quantum-resistant security stack covering key exchange, digital signatures, and authenticated symmetric encryption. The question is no longer whether those building blocks exist — it is whether the IoT industry will use them in time.

How each piece fits depends on the hardware. ML-KEM is already a practical and immediately practical stackable primitive PQC that can be deployed today on mid-range Cortex-M4 with constrained stack limits, making it the most practically useful PQC primitive to date in constrained devices. ML-DSA does the authentication of firmware at the same level. FALCON is supported on M7-class hardware or by a hardware accelerator, where its compact signatures are worth the floating-point price. SLH-DSA should be used in offline signing pipelines, such as build servers, HSMs, where the time to sign is insignificant and only verification runs on the device. ASCON is involved in virtually anything, such as

hardware levels where the PQC algorithms are unable to reach. These assignments, along with the empirical work reviewed here — pqm4 benchmarks, the Lopez *et al.* Raspberry Pi experiments, and the Gómez-Cambronero *et al.* TLS layered analysis — support these assignments with actual numbers, not just theoretical statements.

Its linking factor is timing. The gadgets coming out production lines today will continue to run in 2035. They cannot be patched afterwards in case quantum threats become real soon than anticipated. PQC support design now, not a future design, is merely the right thing to do as an engineering judgment given what we know about the threat schedule. The standards are prepared. The open-source libraries are in place. The standards that inform you of what is possible on your particular hardware are available. Adoption is what remains.

References

[1] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Review*, vol. 41, no. 2, pp. 303–332, 1999, doi: 10.1137/S0036144598347011.

[2] D. J. Bernstein and T. Lange, "Post-quantum cryptography," *Nature*, vol. 549, pp. 188–194, Sep. 2017, doi: 10.1038/nature23461.

[3] D. Apon *et al.*, "Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process," NIST IR 8413, National Institute of Standards and Technology, Jul. 2022. [Online]. Available: https://doi.org/10.6028/NIST.IR.8413

[4] National Institute of Standards and Technology, "Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process," NIST IR 8545, Mar. 2025. [Online]. Available: https://csrc.nist.gov/pubs/ir/8545/final

[5] National Institute of Standards and Technology, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," Federal Information Processing Standard FIPS 203, Aug. 2024. [Online]. Available: https://doi.org/10.6028/NIST.FIPS.203

[6] National Institute of Standards and Technology, "Module-Lattice-Based Digital Signature Standard," Federal Information Processing Standard FIPS 204, Aug. 2024. [Online]. Available: https://doi.org/10.6028/NIST.FIPS.204

[7] National Institute of Standards and Technology, "Stateless Hash-Based Digital Signature Standard," Federal Information Processing Standard FIPS 205, Aug. 2024. [Online]. Available: https://doi.org/10.6028/NIST.FIPS.205

[8] K. McKay and M. S. Turan, "Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions," NIST Special Publication 800-232, National Institute of Standards and Technology, Aug. 2025. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-232

[9] T. Liu, "Post-Quantum Cryptography for Internet of Things: A Survey on Performance and Optimization," *arXiv preprint* arXiv:2401.17538, Jan. 2024. [Online]. Available: https://arxiv.org/abs/2401.17538

[10] J. Lopez, V. Cadena, and M. S. Rahman, "Evaluating Post-Quantum Cryptographic Algorithms on Resource-Constrained Devices," *arXiv preprint* arXiv:2507.08312, Jul. 2025. [Online]. Available: https://arxiv.org/abs/2507.08312

[11] P. Kampanakis, H. Tschofenig, and T. Fossati, "Secure IoT in the Era of Quantum Computers — Where Are the Bottlenecks?" *Sensors*, vol. 22, no. 7, p. 2484, 2022, doi: 10.3390/s22072484. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9003183/

[12] M. J. Kannwischer, M. Krausz, R. Petri, and S.-Y. Yang, "pqm4: Benchmarking NIST Additional Post-Quantum Signature Schemes on ARM Cortex-M4," *IACR ePrint Archive*, Report 2024/112, Jan. 2024. [Online]. Available: https://eprint.iacr.org/2024/112

[13] M. J. Kannwischer, "Implementing Kyber and Dilithium on ARM Cortex-M: NTT Optimizations and Assembly Techniques," Tutorial at *IACR CHES 2023*, Prague, Sep. 2023. [Online]. Available: https://ches.iacr.org/2023/affiliated.php

[14] C. Hamouda *et al.*, "Benchmarking and Analysing the NIST PQC Finalist Lattice-Based Signature Schemes on the ARM Cortex M7," *IACR ePrint Archive*, Report 2022/405, 2022. [Online]. Available: https://eprint.iacr.org/2022/405

[15] T. Pornin, "Falcon on ARM Cortex-M4: An Update," *IACR ePrint Archive*, Report 2025/123, Jan. 2025. [Online]. Available: https://eprint.iacr.org/2025/123

[16] J. Choi *et al.*, "Optimized Falcon Verify on Cortex-M4 for UAV Applications," *ICT Express*, vol. 10, no. 4, pp. 675–692, 2024, doi: 10.1016/j.icte.2024.01.006.

[17] J. Kaur, A. Cintas Canto, M. Mozaffari Kermani, and R. Azarderakhsh, "A Comprehensive Survey on the Implementations, Attacks, and Countermeasures of the Current NIST Lightweight Cryptography Standard," *ACM Computing Surveys*, 2023, doi: 10.1145/3618959. [Online]. Available: https://arxiv.org/abs/2304.06222

[18] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, "ASCON v1.2," Submission to NIST Lightweight Cryptography Standardization, 2021. [Online]. Available: https://ascon.iaik.tugraz.at/

[19] A. Schöpf *et al.*, "The Quest for Efficient ASCON Implementations: A Comprehensive Review of Implementation Strategies and Challenges," *Cryptography*, vol. 4, no. 2, p. 15, 2025, doi: 10.3390/cryptography4020015. [Online]. Available: https://www.mdpi.com/2674-0729/4/2/15

[20] E. Svensén *et al.*, "Comparative Performance Analysis of Lightweight Cryptographic Algorithms on Resource-Constrained IoT Platforms," *Sensors*, 2025. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC12473500/

[21] S. Ravi *et al.*, "The Untapped Potential of Ascon Hash Functions: Benchmarking, Hardware Profiling, and Application Insights for Secure IoT and Blockchain Systems," *MDPI Applied Sciences*, 2025. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC12526793/

[22] D. Gómez-Cambronero, D. Munteanu, and A. I. González-Tablas, "Layered Performance Analysis of TLS 1.3 Handshakes: Classical, Hybrid, and Pure Post-Quantum Key Exchange," *arXiv preprint* arXiv:2603.11006, Mar. 2026. [Online]. Available: https://arxiv.org/abs/2603.11006

[23] R. Sattel, C. Spang, C. Heinz, and A. Koch, "PQC-HA: A Framework for Prototyping and In-Hardware Evaluation of Post-Quantum Cryptography Hardware Accelerators," *arXiv preprint* arXiv:2308.06621, Aug. 2023. [Online]. Available: https://arxiv.org/abs/2308.06621

[24] J. X. Chen *et al.*, "Area and Power Efficient FFT/IFFT Processor for FALCON Post-Quantum Cryptography," *arXiv preprint* arXiv:2401.10591, Jan. 2024. [Online]. Available: https://arxiv.org/abs/2401.10591

[25] C.-L. Chen *et al.*, "Lightweight Post-Quantum Cryptography: Applications and Countermeasures in Internet of Things, Blockchain, and E-Learning," *Engineering Proceedings*, vol. 103, no. 1, p. 14, 2025, doi: 10.3390/engproc103010014. [Online]. Available: https://www.mdpi.com/2673-4591/103/1/14

[26] G. Banegas *et al.*, "A Comparative Study of Post-Quantum Cryptosystems for Internet-of-Things Applications," *Sensors*, vol. 22, no. 3, p. 713, 2022, doi: 10.3390/s22030713. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8779321/

Bibliography / Additional Reading

The following resources were consulted during preparation of this survey and are recommended for further reading:

[A1] Open Quantum Safe Project, `liboqs` open-source library of post-quantum algorithms, https://openquantumsafe.org/.

[A2] NIST PQC Standardization project page, https://csrc.nist.gov/projects/post-quantum-cryptography.

[A3] ASCON official specification and reference code, https://ascon.iaik.tugraz.at/.

[A4] pqm4 benchmarking framework for ARM Cortex-M4, https://github.com/mupq/pqm4.

[A5] D. J. Bernstein *et al.*, "Introduction to Post-Quantum Cryptography," in *Post-Quantum Cryptography*, Springer Berlin, 2009.